CSE 213 — Software Engineering

UML Class Diagrams and ERD-to-Class

Lesson notes · Exam approach · TF practice with solutions · Quick notes (Anindya Sir's portion)

How to Read These Notes

EXAM FOCUS	Exam importance, where it appears in the TF collection
MEMORIZE	Rules, tables and formulas to memorize
TRAP	Common mistakes and exam traps
EXAM-READY ANSWER	Model wording you can write in the exam
NOTE	Extra explanation / clarification
Grey box, monospace	Code, worked tables, derivations and hand-drawn sketches
Centered grey box	Formula to remember / final numerical result
Purple heading / ✓ green	Practice questions / their solutions

Contents

Exam Map — What the TF Papers Ask — class-diagram questions by year
Lesson 9 — Class Notation: Attributes, Operations, Kinds of Classes — Lec 5 sl. 1–5
Lesson 10 — Relationships, Multiplicity, Aggregation vs Composition — Lec 5 sl. 6–14
Lesson 11 — Drawing a Class Diagram from a Scenario (worked TF diagrams) — Lec 5 sl. 15–24
Lesson 12 — From ERD to Class Diagram; CRC Cards — Lec 6 sl. 1–10
Quick Notes — Class Diagrams — notation sheet + key diagrams

NOTE Slide numbers refer to the PDF page numbers of the deck named under each lesson title. TF page numbers refer to 'CSE 213 All Past Papers.pdf' (70 pages); the newest paper (22 batch, labelled 2024-25 here) is a separate file, cited as '22 batch p.N'. Old course code: CSE 307. Lessons 9–11: 'Lec 5 - class diagram-final.pdf' (24 slides; slides 1–14 are identical to 'Lec 4 - class diagram.pdf' and to 'class diagram.pptx'). Lesson 12: 'Lec 6 - ERD to class.pdf'. Diagrams are drawn as text boxes: [Class] blocks, ◆ = composition, ◇ = aggregation, △ = generalisation, multiplicities at each end.

Exam Map — What the TF Papers Ask

Session (paper)	Questions in this note file
2022-23 (CSE 307)	Q4(b) Accounting software: Account / Debtor / Creditor / child & leaf accounts / 2-step authorisation — class diagram (12) [L11]
2021-22 (CSE 307)	Q2(b) University: departments, faculties, head, courses, students — class diagram (10) [L11]
2020-21 (CSE 307)	Q5(a) Media agency newspaper advertisements — class diagram (16) [L11]
2019-20 (CSE 307)	Q5(a) Hospital wards, teams, consultant / junior doctors, patients — class diagram (20) [L11]
2018-19 (CSE 307)	Q1(a) Bangladesh Shipping Management (shipped items, transportation events) — class diagram (14) [L11] · Q4(c) Aggregation and composition — video rental store (8) [L10]
2017-18 (CSE 307)	Q5(a) Hospital wards / teams — class diagram (14) [L11] · Q8(c) Aggregation and composition example diagrams (6) [L10]
2014-15 (CSE 307)	Q8(a) Types of classes in a class diagram (12⅔) [L9] · Q8(b) Music-CD lending database — class and sequence diagrams (22) [L11]
2013-14 (CSE 307)	Q7(a) CSE 308 teams / assignments / midterm — class diagram (20) [L11]
2012-13 (CSE 307)	Q7(c) Library: librarians in shifts, copies, authors, publishers — class diagram (12) [L11] · Q8(b) CRC: meaning and two benefits (5) [L12]
2011-12 (CSE 307)	Q2(d) CRC card and its usage (10) [L12]

EXAM FOCUS A scenario class diagram (10–20 marks) appears in 2022-23, 2021-22, 2020-21, 2019-20, 2018-19 and 2017-18 — every recent paper. The hospital wards & teams scenario (2019-20 Q5(a), 2017-18 Q5(a)) is literally slide 19 of Lec 5. Practise the 6-step method of Lesson 11 on each scenario; aggregation vs composition (2018-19, 2017-18) is a quick 6–8 marks.

Lesson 9 — Class Notation: Attributes, Operations, Kinds of Classes

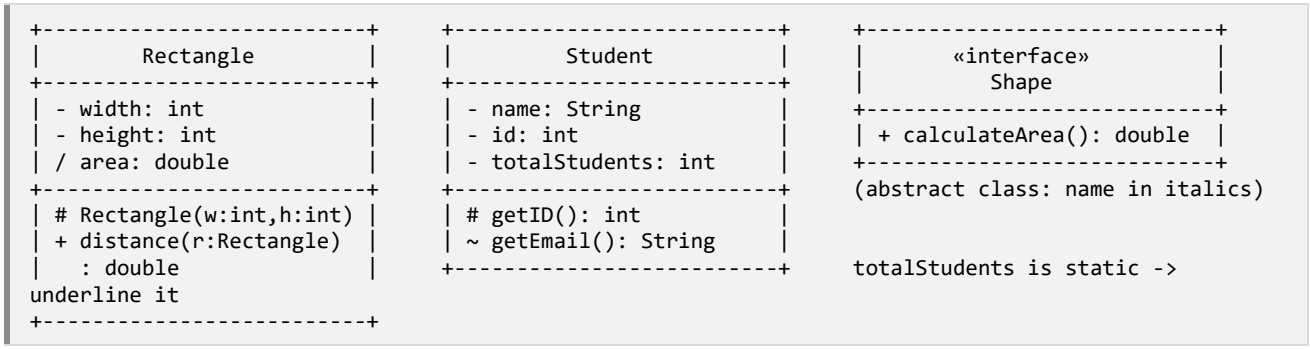
Slides 1–5 (Lec 5 - class diagram-final)

1. What a class diagram shows (sl. 1–2)

A picture of the classes of an OO system: their fields and methods, and the connections between classes that interact or inherit. It does **not** show how classes interact over time (that is a sequence diagram) or algorithms.

Benefits: models data for complex information systems; overview of the structure before reading code (reduces maintenance time); better understanding of the application; detailed charts of the code to be written; helpful for developers and other stakeholders.

2. A single class (sl. 3)



- Name on top; «interface» above an interface name; *italics* for an abstract class.
- Attributes in the middle (optional); operations at the bottom (optional).
- Trivial get/set methods may be omitted, but never omit a method of an interface; do not repeat inherited methods.

3. Attribute and operation syntax (sl. 4–5)

visibility name : type [count] = default_value
visibility name(param: type, …) : return_type

Symbol	Meaning
+	public
#	protected
-	private
~	package (default)
/	derived (computed, e.g. / age from DOB)
underline	static (class-level)

Example (sl. 4–5): ~ courses[100]: Course; / Age: int; + Student(n: String, dob: String) — no return type for constructors or void methods; + calculateGPA(crs: Course[]): float.

4. Kinds of classes (2014-15 Q8(a) — textbook + slides)

Kind	Notation	Example
Concrete class	plain name	Student

Kind	Notation	Example
Abstract class	name in italics (or {abstract})	*Security*, *Payment*, *RentalItem* (sl. 12–13, 16)
Interface	«interface» stereotype; realised with dashed line + hollow triangle	«interface» Shape
Association class	class box joined to an association by a dashed line	Job between Company and Person (sl. 11)
Enumeration	«enumeration» with literal values	Gender {male, female}, OrderStatus (sl. 19, 17)
Stereotyped analysis classes	«entity», «boundary», «control»	Invoice «entity», CreateInvoiceUI «boundary»
Data type / utility	«datatype», «utility» (static members only)	Address «datatype» (sl. 18)

Practice — Lesson 9

(a) Actual — 2014-15 Q8(a). (b) Constructed.

(a) Describe different types of classes used in a class diagram with example. (12⅔)

(b) Write the class box for Account: private balance (double, default 0), static rate, derived interest, public deposit(amount) and withdraw(amount) returning boolean.

✓ Solution

(a) The table in §4 — draw a small box for each kind (concrete, abstract in italics, interface with «interface», association class with dashed link, enumeration, entity/boundary/control stereotypes).

```
+--------------------------------------+
|               Account                |
+--------------------------------------+
| - balance: double = 0                |
| - rate: double            (underlined: static)
| / interest: double                   |
+--------------------------------------+
| + deposit(amount: double)            |
| + withdraw(amount: double): boolean  |
+--------------------------------------+
```

Lesson 10 — Relationships, Multiplicity, Aggregation vs Composition

Slides 6–14 (Lec 5 - class diagram-final)

1. The relationship types (sl. 6–10)

Relationship	Line / arrow	Meaning
Generalisation (class)	solid line, triangle at the parent	inheritance; hierarchies drawn top-down, arrows up
Realisation (interface)	dashed line, hollow triangle	class implements interface
Association	solid line, optional name ▶ and navigability arrow	objects are linked / use each other
Aggregation	hollow diamond ◇ at the whole	'is part of' — part can exist without the whole (Car ◇— Engine)
Composition	filled diamond ◆ at the whole	'is entirely made of' — parts live and die with the whole (Book ◆— Page)
Dependency	dashed line, open arrow	'uses temporarily' — implementation detail, not state (Lottery ⇢ Random)

> **NOTE** Slide 7 states: parent is a class → solid line with **black** arrow; interface → dashed line, white arrow; abstract class → solid line, white arrow. Standard UML uses a hollow triangle for every generalisation; either is accepted if you are consistent.

2. Multiplicity (sl. 8–9)

Written	Meaning
1	exactly one
0..1	zero or one
or 0..	zero or more
1..*	one or more
2..4	between 2 and 4
3..*	three or more

Read each end from the opposite class: Book 1 ◆—— * Page = 'each book has many pages, each page belongs to exactly one book'. Car 1 —— 1 Engine = 'each car has one engine, each engine belongs to one car'.

3. Association class (sl. 11)

When the association itself has properties: Company * —— 1..* Person with a Job class (description, dateHired, salary) hanging on the association by a dashed line. Two separate associations Company–Job and Job–Person would be wrong — they would not tie one Job to one specific pair.

4. Slide examples (sl. 12–14)

```
Video rental store (sl. 12):
  Customer 1 ◇---- RentalInvoice ◆---- 1..* RentalItem {abstract}       RentalInvoice ---- 0..1
CheckoutScreen
                                              △
                                  +-----------+-----------+
                                 DVD          VHS         Game
Inheritance (sl. 13): Security {abstract: presentValue(), history()} <- CashAccount, Stock, Bond,
Property;
                      Stock <- SmallCapStock, LargeCapStock
Multiple inheritance (sl. 14): BankAccount <- InterestBearingItem, InsurableItem;  RealEstate <-
InsurableItem, Asset
```

TRAP Aggregation vs composition is decided by **lifetime and sharing**, not by size. If deleting the whole must delete the part, and the part cannot be shared by two wholes → composition. If the part survives (an engine can be moved to another car; a player stays when the team is dissolved) → aggregation.

Practice — Lesson 10

(a) Actual — 2018-19 Q4(c). (b) Actual — 2017-18 Q8(c).

(a) Show example class diagrams of aggregation and composition in the context of a video rental store. (8)

(b) Show example diagrams of a real-world scenario to present aggregation and composition. (6)

✓ Solution

(a) Composition: a rental invoice is made of its rental line items; deleting the invoice deletes its lines. Aggregation: a customer has invoices / a video store has a catalogue of video titles — the titles exist even if the catalogue section is removed.

```
COMPOSITION                                  AGGREGATION
+---------------+ 1        1..* +-----------+  +-----------+ 1        0..* +---------------+
| RentalInvoice |◆-------------| RentalLine |  | Customer  |◇--------------| RentalInvoice |
+---------------+              +-----------+  +-----------+                +---------------+
| invoiceNo     |              | dueDate   |  | name      |                (an invoice record
outlives
| date          |              | fee       |  | phone     |                 a deleted customer
account)
+---------------+              +-----------+  +-----------+
                                     | * refers to 1                     +-----------+ 1     0..* +--
-------+
                                  +-----------+                          | Catalogue |◇----------|
Video   |
                                  | Video     |                          +-----------+           +---
------+
```

(b) Composition: House 1 ◆—— 1..* Room (rooms are destroyed with the house). Aggregation: Department 1 ◇—— * Professor (professors still exist if the department closes). Mark multiplicities at both ends and the diamond at the whole.

Lesson 11 — Drawing a Class Diagram from a Scenario (worked TF diagrams)

Slides 15–24 (Lec 5 - class diagram-final)

1. Method (used for every solution below)

1. Underline nouns → candidate classes; drop roles that are only attributes (name, date, rate).
2. Find 'is a kind of' → generalisation; 'has / consists of' → aggregation or composition (check lifetime).
3. Other verbs → named associations (treats ▶, teaches ▶, books ▶).
4. Put a multiplicity at **both** ends; read every line aloud in both directions.
5. Add key attributes and 1–3 operations per class; use enumerations for fixed sets (gender, status).
6. Write any rule that UML lines cannot show as a {constraint} note.

2. Slide models to know (sl. 15–24)

- sl. 15 Bank/ATM: Bank 1 ◇ * ATM, Bank 1 ◇ * Customer, Customer 1 — * Account (Current, Saving), ATM transactions.
- sl. 16–17 Order system: Customer 1 — * Order ◆ 1..* OrderDetail * — 1 Item; Order 1 — * Payment {abstract} ← Cash, Check, Credit (and WireTransfer); «enumeration» OrderStatus.
- sl. 18 Library domain: Library, Book, BookItem, Author, Account, Patron/Librarian, Catalog, «datatype» Address.
- sl. 19 Hospital wards and teams (the TF question — solved below). sl. 20 Hospital organisation: Person ← Patient, Staff ← Operations/Administrative/Technical staff ← Doctor, Nurse, Front-desk, Technician …
- sl. 21 Online shopping (WebUser, Customer, Account, ShoppingCart, Order, LineItem, Product, Payment).
- sl. 22 University (School, Department, Student, Instructor, Course) — the 2021-22 question. sl. 23 Student — Enrollment(marks) — Seminar with {ordered, FIFO} waiting list. sl. 24 airline with stereotypes and a note.

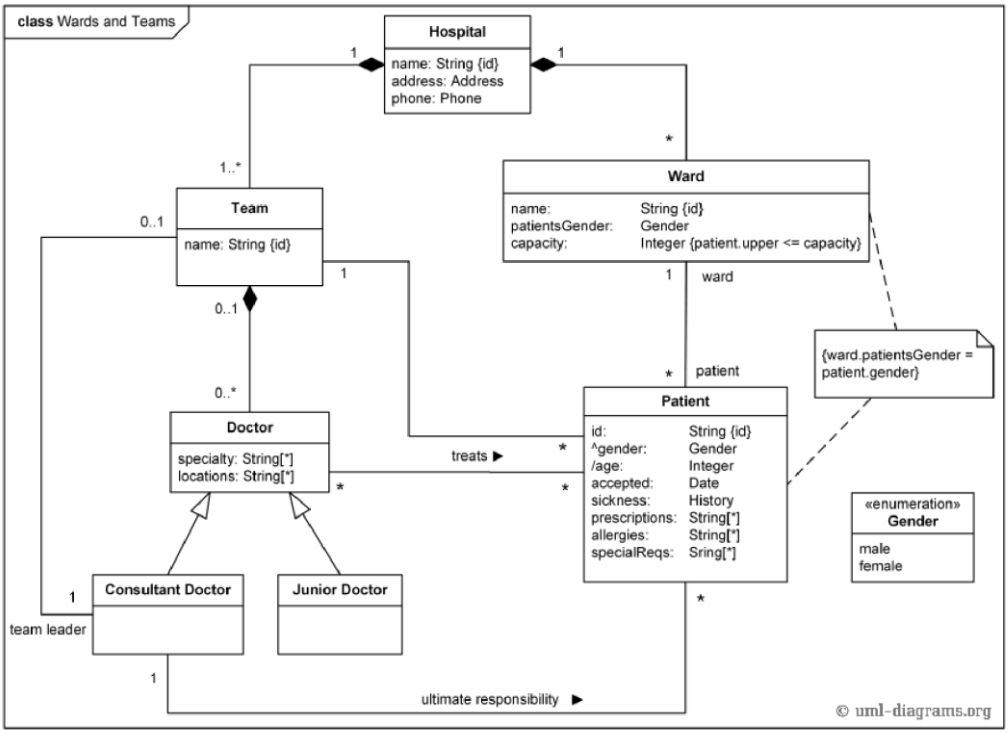


Figure 11.1 — Hospital wards and teams (sl. 19) — answer to 2019-20 Q5(a) / 2017-18 Q5(a)

Practice — Lesson 11

(a) Actual — 2019-20 Q5(a) and 2017-18 Q5(a). (b) Actual — 2021-22 Q2(b). (c) Actual — 2022-23 Q4(b). (d) Actual — 2020-21 Q5(a). (e) Actual — 2018-19 Q1(a). (f) Actual — 2013-14 Q7(a). (g) Actual — 2012-13 Q7(c). (h) Actual — 2014-15 Q8(b) (class part; sequence part in the Interaction Diagrams notes).

(a) Hospital: wards (unique name, gender, fixed capacity, empty or with patients of that gender); doctors in teams (unique name/code) headed by a consultant who supervises fellows/residents/students; the rest are junior doctors; a doctor is in at most one team; consultants visit and treat their patients and advise release. Class diagram. (20 / 14)

(b) University: departments, each with faculties and one head; courses offered by departments (possibly none); students are university members and attend courses instructed by faculties. Attributes and methods; no visibility. (10)

(c) Accounting software: Account (chart of accounts); each transaction debits one account and credits another; made from a user authenticated by two-step authentication (password, OTP); Debtor and Creditor store which account is debited/credited; accounts may have children (leaf = no child); an account with a transaction or a child cannot be deleted. (12)

(d) Media agency orders ads in newspapers; each newspaper has a standard rate per column-inch per page (changes over time) and gives the agency its own discount; expected readership per page; an employee prepares a media plan (page, size, date, newspaper) for a product within a budget; the client approves; another employee generates a Release Order listing ads with positions, dates and cost. (16)

(e) BSM shipping: shipped items (goods or food) with item number, weight, dimensions, insurance amount, destination, final delivery date; received at one retail center; travel via one or more transportation events (schedule number, type flight/truck, delivery route). (14)

(f) CSE 308: students form teams of 2–3, a team completes 0–3 assignments, each student takes exactly one midterm, every assignment and midterm gets a mark. (20)

(g) Library: librarians in shifts, multiple copies per book, one or more authors, one publisher, a user borrows up to 10 books and returns to the librarian on shift. Attributes and associations only. (12)

(h) Music CDs lent to friends: several copies of some CDs, a daily fee per loaned CD; query how many CDs a friend has, how long, how much owed, who has a CD. (12 of 22)

✓ Solution

(a) Classes: Hospital, Ward, Team, Doctor {abstract}, ConsultantDoctor, JuniorDoctor, Patient, «enumeration» Gender. Figure 11.1 is the model answer; text version:

```
Hospital 1 ◆---- 1..* Ward [name {id}, patientsGender: Gender, capacity: int]
Hospital 1 ◆---- 1..* Team [name {id}]                    Ward 1 ---- 0..* Patient  ('on ward')
Team 0..1 ◇---- 1..* Doctor {abstract} [specialty, locations]
Team 0..1 ---- 1 ConsultantDoctor   (team leader / heads)
Doctor <|-- ConsultantDoctor + supervise(), visit(p), prescribe(p, t), adviseRelease(p)
Doctor <|-- JuniorDoctor   (fellow / resident / medical student = role attribute or 3 subclasses)
ConsultantDoctor 1 ---- 0..* JuniorDoctor  (supervises ▶)
Doctor * ---- * Patient  (treats ▶)          ConsultantDoctor 1 ---- * Patient (ultimate responsibility ▶)
Patient [id, name, gender: Gender, admitted: Date, prescriptions: String[*]]
{constraint} ward.patientsGender = patient.gender ;  {constraint} ward.patients->size() <= ward.capacity
«enumeration» Gender { male, female }
```

(b) Matches sl. 22 with the question's wording.

```
University 1 ◆---- 1..* Department [name; addCourse(c), assignHead(f)]
University 1 ◇---- * Student [id, name; enroll(c)]         (students are members)
Department 1 ◇---- 1..* Faculty [name, designation; teach(c)]
Department 1 ---- 1 Faculty   (head ▶)        {head must be one of the department's faculties}
Department 1 ---- 0..* Course [code, title, credit]  (offers ▶ – 0..* because a department may offer
none)
Faculty 1..* ---- * Course (instructs ▶)
Student * ---- 1..* Course (attends ▶)  -- optional association class Enrollment [semester, grade]
```

(c) Account is a Composite-like self-association (parent 0..1 — children 0..*). A Transaction always has exactly one Debtor entry and one Creditor entry, each pointing to an Account. Deletion rules are constraints plus a canDelete() operation.

```
Account [code, name, balance; isLeaf(): boolean; canDelete(): boolean; delete()]
   parent 0..1 ◇---- 0..* children (Account)        isLeaf() = children->isEmpty()
{canDelete() = children->isEmpty() and transactions->isEmpty()}
Transaction [id, date, amount; post()]
Transaction 1 ◆---- 1 Debtor   [amount] ---- 1 Account (debited ▶)       Account 1 ---- 0..* Debtor
Transaction 1 ◆---- 1 Creditor [amount] ---- 1 Account (credited ▶)      Account 1 ---- 0..* Creditor
{debtor.account <> creditor.account ; debtor.amount = creditor.amount}
User [userId, password] 1 ---- 0..* Transaction (makes ▶)
User 1 ---- 1 TwoStepAuthenticator [verifyPassword(pw), sendOTP(), verifyOTP(code)]
```

(d) Rate changes over time → keep a dated Rate class; the discount depends on the newspaper–agency pair → attribute of Newspaper (only one agency) or an association class; readership is per page.

```
Agency 1 ◇---- * Employee [name; preparePlan(), generateReleaseOrder()]
Newspaper [name, agencyDiscount%] 1 ◆---- 1..* Page [pageNo, expectedReadership]
Page 1 ---- 1..* Rate [ratePerColumnInch, effectiveFrom, effectiveTo]   (history of rates)
Client 1 ---- * Product ;  Client 1 ---- * MediaPlan (approves ▶)
MediaPlan [budget, status, totalReadership(); approve()] 1 ◆---- 1..* AdPosition [size(col-inch), date]
AdPosition * ---- 1 Page        cost() = size x rate(on date) x (1 - newspaper.agencyDiscount)
Employee 1 ---- * MediaPlan (prepares ▶)       MediaPlan * ---- 1 Product (for ▶)
ReleaseOrder [orderNo, issueDate; totalCost()] 1 ---- 1 MediaPlan (generated from ▶)
Employee 1 ---- * ReleaseOrder (generates ▶)   ReleaseOrder 1 ---- 1..* AdPosition (lists ▶)
```

(e) ShippedItem is abstract with Goods and Food subclasses; transportation events are many-to-many with items (an event carries many items).

```
ShippedItem {abstract} [itemNumber {id}, weight, dimensions, insuranceAmount, destination,
finalDeliveryDate]
     △ Goods        △ Food
RetailCenter [id, address] 1 ---- 0..* ShippedItem (receives ▶)
ShippedItem 0..* ---- 1..* TransportationEvent [scheduleNumber {id}, type: {flight, truck},
deliveryRoute]
   (travels via ▶; order kept with {ordered})
```

(f)

```
Student [id, name] 2..3 ---- 1 Team [teamNo]       (member of ▶)
Team 1 ---- 0..3 AssignmentSubmission [mark]  ---- 1 Assignment [no, postedDate]
Student 1 ---- 1 MidtermAttempt [mark] ---- 1 Midterm [date]     (each student takes exactly one)
Assignment 1 precedes 0..1 Assignment  {A1 -> A2 -> A3; A3 available only after A2 and midterm}
```

(g)

```
Book [isbn, title] * ---- 1..* Author [name]     Book * ---- 1 Publisher [name]
Book 1 ◆---- 1..* Copy [copyNo, status]
User [id, name] 0..1 ---- 0..10 Copy (borrows ▶)   -- association class Loan [issueDate, dueDate,
returnDate]
Librarian [id, name] * ---- 1 Shift [day, start, end]
Loan * ---- 1 Librarian (issued by ▶)    Loan * ---- 0..1 Librarian (received by ▶ – the one on
shift)
```

(h)

```
Friend [name, phone; cdCount(), amountOwed()] 1 ---- 0..* Loan
CD [title, artist] 1 ◆---- 1..* Copy [copyNo]                Copy 1 ---- 0..* Loan (history)
Loan [startDate, returnDate, feePerDay; daysOut(), fee()]  {at most one open Loan per Copy}
Queries: friend.loans (open) -> count, daysOut, sum(fee); copy.openLoan.friend -> who has it
```

Exam Approach

1. Spend 1 minute listing nouns and verbs from the scenario next to the question.
2. Draw inheritance first (top), then the main whole–part chains, then associations.
3. Never leave an association without multiplicities at both ends.
4. Write constraints the diagram cannot show (capacity, same gender, cannot delete).

Lesson 12 — From ERD to Class Diagram; CRC Cards

Slides 1–10 (Lec 6 - ERD to class)

1. The University of Toronto scenario (sl. 2)

Departments are each managed by a chair and have at least one professor; a professor belongs to one or more departments; each course is taught by at least one professor (a professor on sabbatical may teach none); a course may be taught several times by different professors. Known data: department name, professor name and employee id, course name, schedule, term/year, assignments and offering department.

2. Steps (sl. 3–10)

1. Entity/class candidates (sl. 3): department, chair, professor, course, course section.
2. Relationship matrix (sl. 4) and ER diagram with (min, max) cardinalities (sl. 5–6).
3. Association matrix (sl. 7): mark aggregations.
4. Basic class diagram (sl. 8): Course 0..* — teaches — 0..* Professor; Department 1 ◇ offers 1..* Course; Department 1..* ◇ assigned 1..* Professor; Chair is a Professor; Department 1 — manages — 1 Chair.
5. Add attributes (sl. 9): Course(name, schedule, term), Professor(name, employeeId), Department(name).
6. Add operations (sl. 10): Course.modifySchedule(); Professor.prepareMaterial(), teachLecture(), evaluateAssignments(); Department.addCourse(), addProfessor(), removeCourse(), removeProfessor(), changeChair(); Chair.createPolicy(), evaluateProfessor().

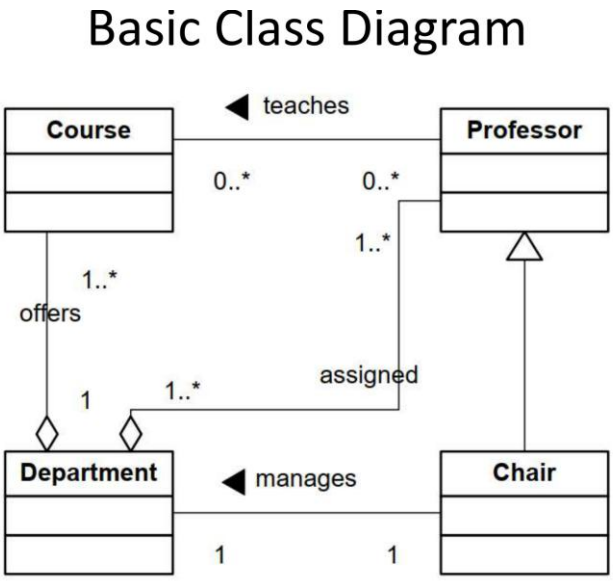


Figure 12.1 — Basic class diagram derived from the ERD (sl. 8)

3. CRC cards (textbook — 2012-13 Q8(b), 2011-12 Q2(d))

CRC = Class–Responsibility–Collaborator. One index card per class: class name on top, its responsibilities (what it knows and does) on the left, its collaborators (classes it needs) on the right. The team role-plays a use case by passing cards around.

```
+-----------------------------------------+
| Class: Order                            |
+-------------------------------+---------+
| Responsibilities              | Collaborators|
| know its line items           | OrderLine   |
| compute total                 | Product     |
| check stock before confirming | Inventory   |
+-------------------------------+---------+
```

Benefits: (1) cheap, informal way to find classes and distribute responsibilities before drawing UML — it keeps classes cohesive (too many responsibilities = split the card); (2) involves users and developers together in walk-throughs of scenarios, revealing missing classes and collaborations early. Usage: brainstorming, role-play of use cases, input to the class diagram.

Practice — Lesson 12

(a) Actual — 2012-13 Q8(b). (b) Actual — 2011-12 Q2(d). (c) Constructed.

(a) What does CRC stand for? Name two benefits of using CRC for class modeling. (5)

(b) What is a CRC card? Discuss its usages. (10)

(c) From the Toronto scenario, why is Chair a subclass of Professor and not a separate entity?

✓ Solution

(a)/(b) §3: definition, card layout, two benefits, usages (identify classes, assign responsibilities, role-play scenarios, team communication, design review).

(c) A chair is a professor with extra responsibilities (createPolicy, evaluateProfessor); every chair has all professor attributes. Generalisation avoids duplicating them, and the 'manages' association (Department 1 — 1 Chair) records which professor currently chairs.

Quick Notes — Class Diagrams

Revision points for Lessons 9–12.

L9 · Notation

- visibility name: type [count] = default · visibility name(p: T): R.
- + public, - private, # protected, ~ package, / derived, underline = static.
- «interface», italics = abstract; don't omit interface methods; no inherited methods.

L10 · Relationships

- Generalisation ▷, realisation dashed ▷, association line, ◇ aggregation, ◆ composition, dashed → dependency.
- Composition = parts die with the whole, not shared. Book ◆ Page; Car ◇ Engine.
- Association class for properties of the link (Job between Company and Person).

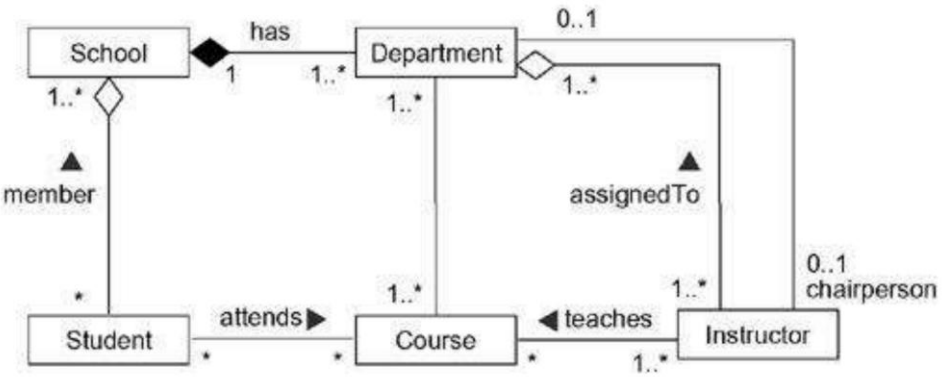
L11 · Scenarios

- Nouns → classes, 'kind of' → ▷, 'has' → ◇/◆, verbs → named associations, multiplicity both ends, constraints.
- Hospital wards/teams = sl. 19; university = sl. 22.

L12 · ERD → class, CRC

- Candidates → relationship matrix → ER (min,max) → class diagram → attributes → operations.
- CRC = Class, Responsibility, Collaborator.

Key Diagrams to Reproduce



22

University (sl. 22)